

Comparative Evaluation of 7 ML Models for Network Intrusion Detection (CSE-CIC-IDS2018)

การเปรียบเทียบโมเดล Machine Learning 7 ตัว สำหรับระบบตรวจจับการบุกรุกเครือข่าย (CSE-CIC-IDS2018)

Bilingual document / เอกสารสองภาษา — Part I is English, Part II is Thai (ภาษาไทย). All numbers come from the run bundle `results/ids2018/300k/` in this repository. ตัวเลขทั้งหมดมาจากผลการรันจริงใน `results/ids2018/300k/` ของโปรเจกต์นี้

PART I — ENGLISH

1. Executive Summary

Seven classical ML models were trained and evaluated on CSE-CIC-IDS2018 under one uniform protocol: an identical 300,000-row stratified sample, an identical stratified 70/30 split, identical preprocessing, and — critically — identical treatment of class imbalance for all seven models.

Headline result table (held-out test partition, 90,000 flows, 15 classes):

#	Model	Accuracy	F1-macro	F1-weighted	Recall-macro	MCC	Binary FPR	Missed attacks (FN)
1	Stacking	0.9839	0.6886	0.9801	0.6806	0.9465	0.00135	834
2	LightGBM	0.9837	0.6870	0.9799	0.6797	0.9460	0.00139	842
3	Random Forest	0.9828	0.6869	0.9786	0.6851	0.9431	0.00199	874
4	XGBoost	0.9837	0.6854	0.9795	0.6796	0.9460	0.00118	860
5	CatBoost	0.9836	0.6764	0.9785	0.6712	0.9454	0.00062	914
6	MLP	0.9816	0.6698	0.9765	0.6728	0.9389	0.00128	1,004
7	Logistic Regression	0.9651	0.6296	0.9592	0.6220	0.8823	0.00815	1,912

Majority-class baseline accuracy = 0.8307 (Benign is 74,764 of 90,000 test flows). Every model clears it comfortably, so unlike the CICIDS2017 study accuracy is not actively misleading here — but it is still uninformative: six of seven models sit inside a 0.23 percentage-point band (0.9816–0.9839). F1-macro is the primary metric.

Three findings that matter more than the ranking

1. The macro-F1 ranking is decided by a single class. Infiltration's F1 ranges from 0.0000 (MLP) to 0.1504 (Stacking), while every other *learnable* class varies by less than 0.02 across all seven models. That one class accounts for 53% of the Stacking–MLP macro-F1 gap and 73% of the Stacking–CatBoost gap. The ranking order of the top six models is, to a first approximation, the ranking order of their Infiltration F1. Model selection on F1-macro is therefore model selection on

one nearly-unlearnable class.

2. Three classes are unlearnable at this sample size, and all seven models score exactly 0.0000 on them. Brute Force -Web (3 test flows), Brute Force -XSS (1) and SQL Injection (1) have too little support to train or to measure. This is a sampling-and-corpus limitation, not a model failure. It also caps the achievable F1-macro at 0.8000 (12/15); the best model reaches 0.6886, i.e. 86% of what is attainable.

3. DoS-SlowHTTPTest and FTP-BruteForce are mutually inseparable in every model. Their F1 scores are ~ 0.59 and ~ 0.79 respectively and agree to three decimal places across architectures as different as a linear model and a stacked ensemble. In the XGBoost confusion matrix, 391 of 776 SlowHTTPTest flows (50.4%) are predicted as FTP-BruteForce. When seven unrelated algorithms make the identical error at the identical rate, the cause is the feature representation, not the learner.

2. Experimental Setup

Item	Value
Corpus	CSE-CIC-IDS2018, 10 daily CSVs, 16,233,002 rows (not the ~ 13 M often cited)
Sample	300,000 rows (1.85% of corpus), stratified, nested-prefix mode, seed 42
Split	Stratified 70/30 $\rightarrow$ 210,000 train / 90,000 test
Classes	15 (1 benign + 14 attack)
Features	84 raw $\rightarrow$ 6 dropped (leakage/identifier) $\rightarrow$ 8 dropped (constant on train) $\rightarrow$ 69 used
Scaler	StandardScaler, fitted on the training split only
Imputation	Training-split median; $\pm\text{inf}$ $\rightarrow$ NaN first
Class weighting	None, for all seven models (see §2.1)
Hyperparameter tuning	None, for all seven models
Seed	42 (single run, no repeats)
Hardware	i7-13620H (10C/16T), 16 GB RAM, RTX 4060 Laptop 8 GB

Columns removed as leakage or identifiers: Flow ID, Src IP, Src Port, Dst IP, Timestamp, Dst Port. Timestamp is a near-perfect label proxy because each attack ran in its own capture window. Columns removed as constant on the training split: Fwd/Bwd Byts b Avg, Fwd/Bwd Pkts b Avg, Fwd/Bwd Blk Rate Avg, Bwd PSH Flags, Bwd URG Flags — CICFlowMeter never populates these.

2.1 Fairness caveats you must read before comparing numbers

Class weighting is disabled for every model, and this was a deliberate methodological choice.

`sklearn.neural_network.MLPClassifier` supports neither `class_weight` nor `sample_weight`. Weighting the six models that do support it would leave MLP as the only model free to favour the majority class, and it would top the accuracy column for that reason alone rather than on merit. A parallel run with `--class-weighting balanced` is preserved in `results/ids2018/300k_balanced_weighting/` for comparison; in that configuration accuracy fell for every weighted model (CatBoost 0.9836 $\rightarrow$ 0.8570, XGBoost 0.9837 $\rightarrow$ 0.9158) while MLP was unchanged, which is precisely the incomparability being avoided here.

LightGBM's `reg_lambda` is set to 1.0, matching XGBoost's default, and this is required rather than tuned. LightGBM defaults `reg_lambda` to 0 while XGBoost defaults it to 1 — so the library defaults are not a fair baseline. Worse, at `reg_lambda=0` LightGBM silently diverges on this dataset: with a single-row SQL Injection class the unregularised optimal leaf value $-g/h$ is unbounded, the softmax overflows, and accuracy peaks at 0.9783 by round 10 then collapses to 0.6777 by round 100, with training stalling at round 59 of 400. No exception is raised. Anyone reproducing this study with stock LightGBM parameters will obtain a broken model that merely looks like a weak one.

Training times are confounded by hardware. XGBoost, CatBoost and the Stacking ensemble's XGBoost base learner ran on CUDA; Random Forest, MLP, Logistic Regression and LightGBM ran on CPU. Train-time comparisons across that boundary are not like-for-like. Inference times are all CPU-side batch predictions on 90,000 rows.

The split is random, not chronological. This is the largest single threat to validity — see §7.

3. Per-Model Technical Evaluation

3.1 Logistic Regression

`solver=lbfgs`, `max_iter=1000`, `C=1.0`. Accuracy 0.9651, F1-macro 0.6296, MCC 0.8823 — last on every quality axis, and the only model whose MCC falls below 0.93.

Its failure is concentrated and diagnosable: 609 false alarms (binary FPR 0.00815, 6.6× CatBoost's) and 1,912 missed attacks (2.3× Stacking's). Bot collapses to F1 0.6061 against 0.99+ for every tree model, which is the clearest evidence that the Bot/Benign boundary is non-linear.

It is nevertheless the extreme point on two operational axes: a 4.5 KB artifact and 6.6M flows/sec, respectively 6,500× smaller and 213× faster than Random Forest. As a pre-filter that cheaply discards obvious traffic it remains defensible; as a primary detector it is not.

> An earlier configuration using `solver=saga`, `max_iter=300` produced accuracy 0.4139 and triggered `ConvergenceWarning`. That number described an unfinished optimisation, not the model, and was discarded.

3.2 Random Forest

300 trees, `max_depth=30`, `max_features=sqrt`. Accuracy 0.9828, F1-macro 0.6869 (3rd), Recall-macro 0.6851 — the best of all seven models.

Random Forest is the recall specialist: it finds more of the rare classes than anything else, including the Stacking ensemble. It pays for that with the highest false-alarm rate among the tree models (149 FPs, binary FPR 0.00199, 3.2× CatBoost's).

Its decisive operational weakness is size: 29.04 MB, the largest artifact by 2× over Stacking and 17.7× over CatBoost, for lower quality than either. Inference is 106,908 flows/sec — respectable, but an order of magnitude below CatBoost.

3.3 XGBoost

400 rounds, `max_depth=8`, `tree_method=hist`, `reg_lambda=1`, `device=cuda`. Accuracy 0.9837, F1-macro 0.6854 (4th), MCC 0.9460.

XGBoost is the most *balanced* model in the field: 2nd-lowest false-alarm rate (88 FPs, binary FPR 0.00118), 860 missed attacks, 119,092 flows/sec, 3.46 MB artifact. It is never the best on any single

axis and never worse than 4th on any either.

It attains F1 1.0000 on both DDOS-HOIC and DoS-Hulk, and 0.9917 on DoS-Slowloris — the best Slowloris score in the field.

3.4 LightGBM

400 rounds, num_leaves=63, reg_lambda=1. Accuracy 0.9837, F1-macro 0.6870 — best of any single model, MCC 0.9460, 842 missed attacks (2nd best).

LightGBM edges out every other standalone model on macro F1, and does so through the class that matters: Infiltration F1 0.1328, second only to the Stacking ensemble's 0.1504 and 7.6× CatBoost's 0.0174.

Its one real weakness is inference throughput: 34,527 flows/sec, the second-slowest in the field and 36× slower than CatBoost, on a 6.43 MB artifact. For batch or near-line analysis this is irrelevant; for inline deployment at high line rates it is disqualifying.

See §2.1 — this model is only usable at all because reg_lambda was corrected.

3.5 CatBoost — RECOMMENDED FOR INLINE DEPLOYMENT

400 iterations, depth=8, MultiClass, task_type=GPU. Accuracy 0.9836, F1-macro 0.6764 (5th), MCC 0.9454.

CatBoost is *fifth* on macro F1 and is still the recommended model, because it dominates every axis a SOC actually pays for:

- Binary FPR 0.00062 — 46 false alarms in 74,764 benign flows. Half the next-best (XGBoost, 88) and 13.2× better than Logistic Regression. At production volumes this is the difference between a triageable queue and alert fatigue.
- 1,248,090 flows/sec — 10.5× XGBoost, 36× LightGBM, 40× Stacking.
- 12.9 s to train — the fastest in the field, 9.2× faster than Stacking.
- 1.64 MB artifact — 17.7× smaller than Random Forest.

Its cost is 914 missed attacks against Stacking's 834, i.e. 80 additional missed flows out of 15,236 attacks (+0.5 pp), and the field's second-worst Infiltration F1 (0.0174). It reaches F1 1.0000 on DDOS-HOIC and SSH-Bruteforce.

The trade is explicit: CatBoost buys a 2× reduction in false alarms and a 10–40× throughput advantage for a 0.5 pp increase in missed attacks.

3.6 Multi-Layer Perceptron (MLP)

hidden=(128, 64), adam, early stopping. Accuracy 0.9816, F1-macro 0.6698 (6th), MCC 0.9389.

The MLP is the only model that scores 0.0000 on Infiltration — it never predicts the class at all. It also has the worst macro precision in the field (0.6742) and 1,004 missed attacks, the worst of any model except Logistic Regression.

It is, however, the second-fastest at inference (1,542,416 flows/sec) on a 0.21 MB artifact, and its 51 s training time is entirely CPU-side — the only model here that could be moved to a GPU for a further speedup.

Its results are identical to the run under balanced class weighting, which is expected and serves as a useful determinism check on the pipeline: MLP is the one model that configuration never touched.

3.7 Stacking Ensemble

RF(150) + LightGBM(200) + XGBoost(200) → Logistic Regression meta-learner, cv=3. Accuracy 0.9839 and F1-macro 0.6886 — best in field on both, MCC 0.9465, 834 missed attacks (best).

The ensemble wins, and the margin is negligible. Against LightGBM alone it buys +0.0016 F1-macro and 8 fewer missed attacks for 5.1× the training time (118.2 s vs 23.1 s) and the worst inference throughput in the field (30,881 flows/sec). Against CatBoost it buys +0.0122 F1-macro for 9.2× the training time and a 40× throughput penalty.

Its genuine contribution is Infiltration F1 0.1504 — the best measured, and 12% better than the best single model. If Infiltration detection is the objective, the ensemble is justified. If overall accuracy is the objective, it is not.

> The ensemble required a compatibility fix to run: `StackingClassifier` refits base learners on CV folds, and with a single-row SQL Injection class most folds omit it, leaving `y` with a gap (0..12, 14). XGBoost rejects non-contiguous labels outright, killing the whole ensemble. `LabelSafeXGBClassifier` in `src/ids2018/models.py` re-encodes labels densely for fitting while reporting the original codes in `classes_`, so scikit-learn pads the missing probability column itself.

4. Consolidated Trade-Off Matrix

4.1 Master comparison

Model	Accuracy	F1-macro	Recall-macro	Precision-macro	MCC	Infiltration F1
Stacking	0.9839	0.6886	0.6806	0.7203	0.9465	0.1504
LightGBM	0.9837	0.6870	0.6797	0.7174	0.9460	0.1328
Random Forest	0.9828	0.6869	0.6851	0.7172	0.9431	0.1037
XGBoost	0.9837	0.6854	0.6796	0.7150	0.9460	0.0963
CatBoost	0.9836	0.6764	0.6712	0.7132	0.9454	0.0174
MLP	0.9816	0.6698	0.6728	0.6742	0.9389	0.0000
Logistic Regression	0.9651	0.6296	0.6220	0.6599	0.8823	0.0193

4.2 Operational cost comparison

Model	Train (s)	Device	Inference (flows/s)	Artifact (MB)	False alarms (FP)	Missed attacks (FN)
CatBoost	12.9	GPU	1,248,090	1.64	46	914
Random Forest	20.4	CPU	106,908	29.04	149	874
LightGBM	23.1	CPU	34,527	6.43	104	842
XGBoost	29.2	GPU	119,092	3.46	88	860
MLP	51.0	CPU	1,542,416	0.21	96	1,004

Class	Support	Stacking	LightGBM	RF	XGBoost	CatBoost	MLP	LogReg
Brute Force -XSS	1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
SQL Injection	1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

5.2 High-volume attacks — solved

DDOS-HOIC, DDoS-LOIC-HTTP, DoS-Hulk, SSH-Bruteforce and DoS-GoldenEye are at or above F1 0.99 for all six non-linear models, with four exact 1.0000 scores. Bot reaches 0.99+ for all tree models. These volumetric and brute-force families are fully separable in CICFlowMeter features and are not a useful basis for model selection — every model gets them right.

5.3 The three unlearnable classes

Brute Force -Web (3 test flows), Brute Force -XSS (1) and SQL Injection (1) score exactly 0.0000 for all seven models. At the corpus level these classes hold 611, 230 and 87 rows respectively out of 16.2M; proportional stratified sampling at 1.85% yields 11, 4 and 2 rows, and a 70/30 split leaves 1–3 in test.

No conclusion about model quality may be drawn from these three rows. They should be reported as "insufficient support" rather than as a score of zero. Raising `--min-per-class` would give them usable support but would break the proportional stratification that the rest of the study depends on.

5.4 Infiltration — the class that decides the ranking

Model	Infiltration F1	F1-macro	Rank agreement
Stacking	0.1504	0.6886	1 / 1
LightGBM	0.1328	0.6870	2 / 2
Random Forest	0.1037	0.6869	3 / 3
XGBoost	0.0963	0.6854	4 / 4
CatBoost	0.0174	0.6764	5 / 5
MLP	0.0000	0.6698	6 / 6

For the six non-linear models the Infiltration ranking and the macro-F1 ranking are identical. Infiltration accounts for 53% of the Stacking–MLP macro-F1 gap and 73% of the Stacking–CatBoost gap.

The underlying failure is visible in the XGBoost confusion matrix: 6,490 of 74,764 Benign flows (8.7%) are predicted as Infiltration, and 518 of 898 true Infiltration flows are predicted as Benign. Precision reaches only 0.055 at recall 0.422 in the class-weighted configuration. Infiltration traffic in CSE-CIC-IDS2018 is, at flow-feature level, indistinguishable from ordinary internal traffic — a limitation of the capture, widely reported in the literature.

5.5 The SlowHTTPTest / FTP-BruteForce collision

DoS-SlowHTTPTest peaks at F1 0.6080 (MLP) and FTP-BruteForce at 0.7884 (Stacking/XGBoost); across all seven models the spread is 0.029 and 0.045 respectively. The XGBoost confusion matrix

shows 391 of 776 SlowHTTPTest flows predicted as FTP-BruteForce and 126 of 1,072 FTP-BruteForce flows predicted as SlowHTTPTest.

Both attacks generate large numbers of short-lived connections to a single service, and after `Dst Port` and `Timestamp` are removed as leakage, the remaining flow statistics do not separate them. Retaining `Dst Port` would separate the two almost perfectly — and would be leakage, because it encodes the lab setup rather than attack behaviour.

> No SHAP analysis was performed for this dataset. The cross-model agreement evidence above is confusion-matrix-based, not attribution-based.

6. Final Deployment Recommendation

There is no single winner across all axes. The recommendation is therefore stated per axis, with the trade explicit.

Primary inline detector: CatBoost. 46 false alarms (binary FPR 0.00062, best in field by 2×), 1,248,090 flows/sec (best among accurate models by 10×), 12.9 s to train, 1.64 MB artifact. Its cost is 80 additional missed attacks relative to Stacking (+0.5 pp of all attack flows) and near-zero Infiltration detection — which, given that the best model in the field reaches only F1 0.1504 on that class, is not a capability any model here actually provides.

Balanced alternative / CPU-only sites: XGBoost. Second-lowest false-alarm rate (88 FPs), 4th on F1-macro, 119,092 flows/sec, 3.46 MB. Never best on any axis, never below 4th on any.

Batch or forensic analysis where throughput is irrelevant: LightGBM. Best single-model F1-macro (0.6870) and second-best Infiltration F1 (0.1328). Its 34,527 flows/sec rules it out for inline use.

Use Stacking only if Infiltration is the explicit target. It is best in field on accuracy, F1-macro and missed attacks, but the margins over LightGBM are +0.0002, +0.0016 and 8 flows respectively, for 5.1× the training cost and the worst throughput measured.

Do not deploy Logistic Regression as a primary detector. 609 false alarms (6.6× the recommended model) and 1,912 missed attacks (2.3× the best). It remains viable as a cheap pre-filter at 6.6M flows/sec and 4.5 KB.

Do not rely on any model in this study for Brute Force -Web, Brute Force -XSS, SQL Injection, or Infiltration. The first three have no measurable support; the fourth is not learnable from these features. Cover these families with signature-based or host-based controls.

7. Limitations & Threats to Validity

1. The split is random, not chronological. This is the most serious limitation and the largest difference from the companion CICIDS2017 study, which used a strict leakage-free chronological protocol. A random stratified split allows flows from the same attack burst — often near-duplicates generated seconds apart by the same tool against the same target — to appear in both training and test. Reported scores are therefore optimistic and should be read as an upper bound. They measure interpolation within a capture, not generalisation to future traffic.

2. Duplicate flows were not removed. `--drop-duplicates` is off by default because enabling it changes the final row count and breaks the exact 300,000/210,000/90,000 structure. Combined with limitation 1, exact duplicate flows spanning the train/test boundary will inflate every score reported here.

3. Three classes have 1–3 test flows. Brute Force -Web, Brute Force -XSS and SQL Injection cannot support any inference. Their 0.0000 scores are artefacts of support, not measurements of capability.
4. No hyperparameter tuning was performed for any model. This is a fairness gain — no model was favoured — but it means every reported score is a floor, and the ranking may not survive per-model tuning. It also means the results are not comparable to published figures obtained with tuned models.
5. Training-time comparison is confounded by hardware. XGBoost, CatBoost and Stacking used CUDA; Random Forest, LightGBM, MLP and Logistic Regression used CPU. See §2.1.
6. Single dataset, single sample, single split, single seed. All results are `RANDOM_STATE=42` on one 300,000-row stratified draw. No cross-validation, no repeated sampling, and therefore no confidence intervals. Differences below ~ 0.005 in F1-macro — which covers the top four models — should not be treated as real.
7. The sample is 1.85% of the corpus. Whether the ranking holds at 500,000 or 1,000,000 rows is untested. The sampling design is nested (300k $\square$ 500k $\square$ 1M), so this can be measured directly by re-running with `--sample-size`.
8. No SHAP or feature-attribution analysis was performed, so claims about **why** models agree rest on confusion-matrix evidence alone.
9. LightGBM's result depends on a non-default parameter. See §2.1. Reproduction with library defaults yields a silently broken model.

PART II — ภาษาไทย

1. บทสรุปผู้บริหาร

โมเดล Machine Learning แบบดั้งเดิม 7 ตัวถูกฝึกและประเมินบน CSE-CIC-IDS2018

ภายใต้โปรโตคอลเดียวกันทั้งหมด: ใช้ตัวอย่างแบบ stratified 300,000 แถวชุดเดียวกัน แบ่ง 70/30 แบบ stratified เหมือนกัน ผ่าน preprocessing เดียวกัน และที่สำคัญที่สุดคือ

จัดการความไม่สมดุลของคลาสด้วยวิธีเดียวกันทั้ง 7 ตัว

ตารางผลลัพธ์หลัก (ชุดทดสอบที่กันไว้ 90,000 flow, 15 คลาส):

#	โมเดล	Accuracy	F1-macro	F1-weighted	Recall-macro	MCC	Binary FPR	โจมตีที่พลาด (FN)
1	Stacking	0.9839	0.6886	0.9801	0.6806	0.9465	0.00135	834
2	LightGBM	0.9837	0.6870	0.9799	0.6797	0.9460	0.00139	842
3	Random Forest	0.9828	0.6869	0.9786	0.6851	0.9431	0.00199	874
4	XGBoost	0.9837	0.6854	0.9795	0.6796	0.9460	0.00118	860
5	CatBoost	0.9836	0.6764	0.9785	0.6712	0.9454	0.00062	914
6	MLP	0.9816	0.6698	0.9765	0.6728	0.9389	0.00128	1,004
7	Logistic Regression	0.9651	0.6296	0.9592	0.6220	0.8823	0.00815	1,912

Accuracy ของการเดาคลาสส่วนใหญ่ (majority-class baseline) = 0.8307 (Benign มี 74,764 จาก 90,000 flow) ทุกโมเดลสูงกว่านี้อย่างชัดเจน ต่างจากกรณี CICIDS2017 ที่ accuracy ทำให้เข้าใจผิดได้ — แต่ที่นี่

accuracy ก็ยังไม่มีประโยชน์ในการตัดสินใจ เพราะ 6 ใน 7 โมเดลอยู่ในช่วงแคบเพียง 0.23 จุดเปอร์เซ็นต์ (0.9816–0.9839) จึงใช้ F1-macro เป็นตัวชี้วัดหลัก

สามข้อค้นพบที่สำคัญกว่าอันดับ

1. อันดับของ F1-macro ถูกตัดสินด้วยคลาสเดียว คลาส Infiltration มี F1 ตั้งแต่ 0.0000 (MLP) ถึง 0.1504 (Stacking) ในขณะที่คลาสอื่นที่ *เรียนรู้ได้* ทุกคลาสต่างกันไม่ถึง 0.02 ระหว่างโมเดลทั้ง 7 ตัว คลาสเดียวนี้อธิบายช่องว่าง F1-macro ระหว่าง Stacking กับ MLP ได้ 53% และระหว่าง Stacking กับ CatBoost ได้ 73% พูดย่อคือการเลือกโมเดลด้วย F1-macro ก็คือการเลือกโมเดลด้วยคลาสเดียวที่แทบเรียนรู้ไม่ได้

2. มี 3 คลาสที่เรียนรู้ไม่ได้เลยที่ขนาดตัวอย่างนี้ และทั้ง 7 โมเดลได้ 0.0000 เท่ากันหมด ได้แก่ Brute Force -Web (3 flow ในชุดทดสอบ), Brute Force -XSS (1) และ SQL Injection (1) ซึ่งมีข้อมูลน้อยเกินกว่าจะฝึกหรือวัดผลได้ นี่เป็นข้อจำกัดของการสุ่มตัวอย่างและตัวชุดข้อมูลเอง ไม่ใช่ความล้มเหลวของโมเดล และยังทำให้ F1-macro สูงสุดที่เป็นไปได้ถูกจำกัดไว้ที่ 0.8000 (12/15) โมเดลที่ดีที่สุดได้ 0.6886 คิดเป็น 86% ของเพดานที่ทำได้จริง

3. DoS-SlowHTTPTest กับ FTP-BruteForce แยกจากกันไม่ออกในทุกโมเดล ค่า F1 อยู่ที่ประมาณ 0.59 และ 0.79 ตามลำดับ และตรงกันถึงทศนิยม 3 ตำแหน่งข้ามสถาปัตยกรรมที่ต่างกันสุดขั้ว ตั้งแต่โมเดลเชิงเส้นไปจนถึง ensemble แบบซ้อนชั้น ใน confusion matrix ของ XGBoost มี 391 จาก 776 flow ของ SlowHTTPTest (50.4%) ถูกทำนายเป็น FTP-BruteForce เมื่ออัลกอริทึม 7 ตัวที่ไม่เกี่ยวข้องกันเลยผิดแบบเดียวกันในอัตราเดียวกัน สาเหตุอยู่ที่การแทนคุณลักษณะ ไม่ใช่ตัวโมเดล

2. การตั้งค่าการทดลอง

รายการ	ค่า
ชุดข้อมูล	CSE-CIC-IDS2018, CSV รายวัน 10 ไฟล์, 16,233,002 แถว (ไม่ใช่ ~13M อย่างที่มักอ้างกัน)
ตัวอย่าง	300,000 แถว (1.85% ของทั้งหมด), stratified, โหมด nested, seed 42
การแบ่ง	Stratified 70/30 → ฝึก 210,000 / ทดสอบ 90,000
คลาส	15 (ปกติ 1 + โจมตี 14)
คุณลักษณะ	84 คอลัมน์ดิบ → ตัด 6 (leakage/identifier) → ตัด 8 (ค่าคงที่) → ใช้จริง 69
Scaler	StandardScaler เรียนรู้จากชุดฝึกเท่านั้น
การเติมค่าว่าง	ค่ามัธยฐานของชุดฝึก โดยแปลง ±inf เป็น NaN ก่อน
การถ่วงน้ำหนักคลาส	ไม่ถ่วง ทั้ง 7 โมเดล (ดู §2.1)
การจูน hyperparameter	ไม่จูน ทั้ง 7 โมเดล
Seed	42 (รันครั้งเดียว ไม่ทำซ้ำ)
ฮาร์ดแวร์	i7-13620H (10C/16T), RAM 16 GB, RTX 4060 Laptop 8 GB

คอลัมน์ที่ตัดทิ้งเพราะเป็น leakage หรือตัวระบุตัวตน: Flow ID, Src IP, Src Port, Dst IP, Timestamp, Dst Port โดย Timestamp เป็นตัวแทนของ label ที่เกือบสมบูรณ์แบบ เพราะการโจมตีแต่ละแบบถูกรันคนละช่วงเวลา ส่วนคอลัมน์ที่ตัดเพราะเป็นค่าคงที่ในชุดฝึก ได้แก่ Fwd/Bwd Byts b Avg, Fwd/Bwd Pkts b Avg, Fwd/Bwd Blk Rate Avg, Bwd PSH Flags, Bwd URG Flags ซึ่ง CICFlowMeter ไม่เคยเติมค่าให้

2.1 ข้อควรระวังเรื่องความเป็นธรรม ที่ต้องอ่านก่อนเปรียบเทียบตัวเลข

ปิดการถ่วงน้ำหนักคลาสในทุกโมเดล และเป็นการตัดสินใจเชิงระเบียบวิธีโดยเจตนา เพราะ sklearn.neural_network.MLPClassifier ไม่รองรับทั้ง class_weight และ sample_weight ถ้าถ่วงน้ำหนักเฉพาะ 6 โมเดลที่ทำได้ MLP จะกลายเป็นโมเดลเดียวที่มีอิสระเอนเข้าหาคลาสส่วนใหญ่ และจะขึ้นน้ำหนักในคอลัมน์ accuracy ด้วยเหตุผลนั้นเพียงอย่างเดียว ไม่ใช่เพราะเก่งกว่าจริง ผลการรันคู่ขนานด้วย --class-weighting balanced เก็บไว้ที่ results/ids2018/300k_balanced_weighting/ ซึ่งในคอนฟิกนั้น accuracy ลดลงในทุกโมเดลที่ถูกถ่วงน้ำหนัก (CatBoost 0.9836 → 0.8570, XGBoost 0.9837 → 0.9158) ขณะที่ MLP ไม่เปลี่ยนแปลง — ซึ่งคือความไม่เท่าเทียมที่เรากำลังหลีกเลี่ยงพอดี

reg_lambda ของ LightGBM ถูกตั้งเป็น 1.0 ให้ตรงกับค่าเริ่มต้นของ XGBoost ซึ่งเป็นสิ่งจำเป็น ไม่ใช่การจูน LightGBM ตั้ง reg_lambda เริ่มต้นเป็น 0 ส่วน XGBoost ตั้งเป็น 1 ค่าเริ่มต้นของทั้งสองไลบรารีจึงไม่ใช่ฐานเปรียบเทียบที่เป็นธรรม และที่แย่กว่านั้นคือ ที่ reg_lambda=0 LightGBM จะ diverge แบบเงิบ ๆ บนชุดข้อมูลนี้: เมื่อคลาส SQL Injection มีเพียงแถวเดียว ค่า leaf ที่เหมาะสมที่สุด -g/h จะไม่มีขอบเขต ทำให้ softmax overflow — accuracy ขึ้นสูงสุด 0.9783 ที่ round 10 แล้วร่วงเหลือ 0.6777 ที่ round 100 และการฝึกหยุดที่ round 59 จาก 400 โดยไม่มี error ใด ๆ ปรากฏ ใครก็ตามที่ทำทำงานนี้ด้วยค่าเริ่มต้นของ LightGBM จะได้โมเดลที่พังแต่ดูเหมือนแค่ "ไม่เก่ง"

เวลาฝึกถูกรบกวนจากความต่างของฮาร์ดแวร์ XGBoost, CatBoost และ base learner ที่เป็น XGBoost ใน Stacking รันบน CUDA ส่วน Random Forest, MLP, Logistic Regression และ LightGBM รันบน CPU การเทียบเวลาฝึกข้ามเส้นแบ่งนี้จึงไม่ใช่การเทียบแบบต่อแบบ ส่วนเวลา inference วัดบน CPU ทั้งหมดด้วยการทำนายแบบ batch บน 90,000 แถว

การแบ่งข้อมูลเป็นแบบสุ่ม ไม่ใช่ตามลำดับเวลา นี่คือภัยคุกคามต่อความถูกต้องที่ใหญ่ที่สุด ดู §7

3. การประเมินเชิงเทคนิครายโมเดล

3.1 Logistic Regression (การถดถอยโลจิสติก)

solver=lbfgs, max_iter=1000, C=1.0 — Accuracy 0.9651, F1-macro 0.6296, MCC 0.8823
อยู่อันดับสุดท้ายในทุกแกนคุณภาพ และเป็นโมเดลเดียวที่ MCC ต่ำกว่า 0.93

ความล้มเหลวของมันเป็นกระจกตัวและวินิจัยได้ชัด: เดือนผิด 609 ครั้ง (binary FPR 0.00815 สูงกว่า CatBoost 6.6 เท่า) และพลาดการโจมตี 1,912 ครั้ง (2.3 เท่าของ Stacking) คลาส Bot ร่วงเหลือ F1 0.6061 เทียบกับ 0.99+ ของโมเดลต้นไม้ทุกตัว ซึ่งเป็นหลักฐานชัดที่สุดว่าเส้นแบ่งระหว่าง Bot กับ Benign ไม่เป็นเชิงเส้น

อย่างไรก็ตามมันเป็นจุดสุดขั้วในสองแกนเชิงปฏิบัติการ: ไฟล์ขนาด 4.5 KB และความเร็ว 6.6 ล้าน flow/วินาที เล็กกว่าและเร็วกว่า Random Forest ถึง 6,500 เท่าและ 213 เท่าตามลำดับ

ใช้เป็นตัวกรองขั้นแรกเพื่อคัดตราฟฟิกที่ชัดเจนออกไปยังพอมมีเหตุผล แต่ใช้เป็นตัวตรวจจับหลักไม่ได้

> คอนฟิกก่อนหน้านี้ใช้ solver=saga, max_iter=300 ให้ accuracy 0.4139 พร้อม ConvergenceWarning ตัวเลขนั้นอธิบายการ optimize ที่ยังไม่เสร็จ ไม่ได้อธิบายตัวโมเดล จึงถูกทิ้งไป

3.2 Random Forest (ป่าสุ่ม)

300 , max_depth=30, max_features=sqrt — Accuracy 0.9828, F1-macro 0.6869 (อันดับ 3), Recall-macro 0.6851 ซึ่งดีที่สุดในบรรดา 7 โมเดล

Random Forest คือผู้เชี่ยวชาญด้าน recall: มันหาคลาสหายากเจอมากกว่าโมเดลใด ๆ รวมถึง Stacking ensemble ด้วย แต่ต้องแลกกับอัตราการเดือนผิดที่สูงที่สุดในกลุ่มโมเดลต้นไม้ (149 FP, binary FPR 0.00199 สูงกว่า CatBoost 3.2 เท่า)

จุดอ่อนเชิงปฏิบัติการที่ชี้ขาดคือขนาด: 29.04 MB ใหญ่ที่สุด มากกว่า Stacking 2 เท่าและมากกว่า CatBoost 17.7 เท่า ทั้งที่คุณภาพต่ำกว่าทั้งคู่ ความเร็ว inference อยู่ที่ 106,908 flow/วินาที ซึ่งพอใช้ได้ แต่ต่ำกว่า CatBoost หนึ่งในระดับ

3.3 XGBoost

400 rounds, max_depth=8, tree_method=hist, reg_lambda=1, device=cuda — Accuracy 0.9837, F1-macro 0.6854 (อันดับ 4), MCC 0.9460

XGBoost เป็นโมเดลที่ สมดุลที่สุด ในสนาม: อัตราเดือนผิดพลาดเป็นอันดับสอง (88 FP, binary FPR 0.00118), พลาดการโจมตี 860 ครั้ง, 119,092 flow/วินาที, ไฟล์ 3.46 MB มันไม่เคยดีที่สุดในแกนใดเลย แต่ก็ไม่เคยแย่กว่าอันดับ 4 ในแกนใดเช่นกัน

ได้ F1 1.0000 ทั้ง DDOS-HOIC และ DoS-Hulk และได้ 0.9917 บน DoS-Slowloris ซึ่งเป็นคะแนน Slowloris ที่ดีที่สุดในสนาม

3.4 LightGBM

400 rounds, num_leaves=63, reg_lambda=1 — Accuracy 0.9837, F1-macro 0.6870

ดีที่สุดในบรรดาโมเดลเดียว, MCC 0.9460, พลาดการโจมตี 842 ครั้ง (ดีเป็นอันดับ 2)

LightGBM เอาชนะโมเดลเดี่ยวอื่นทั้งหมดในด้าน macro F1 และชนะผ่านคลาสที่สำคัญจริง ๆ ด้วย: Infiltration F1 0.1328 เป็นรองแค่ Stacking ensemble ที่ 0.1504 และดีกว่า CatBoost ที่ 0.0174 ถึง 7.6 เท่า

จุดอ่อนที่แท้จริงมีข้อเดียวคือความเร็ว inference: 34,527 flow/วินาที เข้าเป็นอันดับสองจากท้าย และช้ากว่า CatBoost 36 เท่า บนไฟล์ขนาด 6.43 MB สำหรับงานวิเคราะห์แบบ batch เรื่องนี้ไม่สำคัญ แต่สำหรับการติดตั้งแบบ inline ที่ความเร็วสายสูง ถือว่าดกอบ

ดู §2.1 — โมเดลนี้ใช้งานได้ก็เพราะแก้ reg_lambda แล้วเท่านั้น

3.5 CatBoost — แนะนำสำหรับการติดตั้งแบบ inline

400 iterations, depth=8, MultiClass, task_type=GPU — Accuracy 0.9836, F1-macro 0.6764 (อันดับ 5), MCC 0.9454

CatBoost อยู่ อันดับ 5 ในด้าน macro F1 แต่ยังเป็นโมเดลที่แนะนำ เพราะมันเหนือกว่าในทุกแกนที่ SOC จ่ายเงินจริง:

- Binary FPR 0.00062 — เดือนผิดพลาดเพียง 46 ครั้งจาก 74,764 flow ปกติ ครึ่งหนึ่งของอันดับถัดไป (XGBoost 88 ครั้ง) และดีกว่า Logistic Regression 13.2 เท่า ที่ปริมาณระดับ production นี่คือการต่างระหว่างคิวที่คัดกรองไหวกับ alert fatigue
- 1,248,090 flow/วินาที — เร็วกว่า XGBoost 10.5 เท่า, LightGBM 36 เท่า, Stacking 40 เท่า
- ผีเสร็จใน 12.9 วินาที — เร็วที่สุดในสนาม เร็วกว่า Stacking 9.2 เท่า
- ไฟล์ 1.64 MB — เล็กกว่า Random Forest 17.7 เท่า

ต้นทุนคือพลาดการโจมตี 914 ครั้งเทียบกับ 834 ของ Stacking หรือพลาดเพิ่ม 80 flow จากการโจมตีทั้งหมด 15,236 flow (+0.5 จุดเปอร์เซ็นต์) และ Infiltration F1 แย่เป็นอันดับสองจากท้าย (0.0174) ส่วนคลาส DDOS-HOIC และ SSH-Bruteforce ได้ F1 1.0000

การแลกเปลี่ยนชัดเจน: CatBoost ช้อกการลดการเดือนผิดพลาดครึ่งหนึ่งและความเร็วที่มากกว่า 10–40 เท่า ด้วยราคาคือการพลาดการโจมตีเพิ่มขึ้น 0.5 จุดเปอร์เซ็นต์

3.6 Multi-Layer Perceptron (MLP — โครงข่ายประสาทหลายชั้น)

hidden=(128, 64), adam, early stopping — Accuracy 0.9816, F1-macro 0.6698 (อันดับ 6), MCC 0.9389

MLP เป็นโมเดลเดียวที่ได้ 0.0000 บน Infiltration คือไม่เคยทำนายคลาสได้เลยแม้แต่ครั้งเดียว และยังมี macro precision แย่ที่สุดในสนาม (0.6742) กับพลาดการโจมตี 1,004 ครั้ง แย่ที่สุดรองจาก Logistic Regression

แต่มันเร็วเป็นอันดับสองในการ inference (1,542,416 flow/วินาที) บนไฟล์เพียง 0.21 MB และเวลาฝึก 51 วินาทีเป็นการรันบน CPU ล้วน — เป็นโมเดลเดียวในชุดนี้ที่ย้ายไป GPU เพื่อเร่งความเร็วเพิ่มได้

ผลของมันเท่ากันเป๊ะกับรอบที่ถ่วงน้ำหนักคลาส ซึ่งเป็นสิ่งที่คาดไว้ และใช้เป็นการตรวจสอบ determinism ของ pipeline ได้ดี เพราะ MLP เป็นโมเดลเดียวที่คอนฟิกนั้นไม่เคยแตกต่าง

3.7 Stacking Ensemble (การรวมโมเดลแบบซ้อนชั้น)

RF(150) + LightGBM(200) + XGBoost(200) → Logistic Regression meta-learner, cv=3 — Accuracy 0.9839 และ F1-macro 0.6886 ดีที่สุดในสนามทั้งสองค่า, MCC 0.9465, พลาดการโจมตี 834 ครั้ง (ดีที่สุด)

Ensemble ชะจริง แต่ระยะห่างแทบไม่มีความหมาย เทียบกับ LightGBM ตัวเดียว มันซื้อ F1-macro เพิ่ม 0.0016 และพลาดการโจมตีน้อยลง 8 ครั้ง ด้วยราคา เวลาฝึก 5.1 เท่า (118.2 วินาที เทียบกับ 23.1 วินาที) และความเร็ว inference แย่ที่สุดในสนาม (30,881 flow/วินาที) เทียบกับ CatBoost มันซื้อ F1-macro เพิ่ม 0.0122 ด้วยเวลาฝึก 9.2 เท่าและความเร็วที่ลดลง 40 เท่า

คุณค่าที่แท้จริงของมันคือ Infiltration F1 0.1504 ซึ่งดีที่สุดเท่าที่วัดได้ และดีกว่าโมเดลเดียวที่ดีที่สุด 12% ถ้าเป้าหมายคือการตรวจจับ Infiltration ก็มีเหตุผลพอที่จะใช้ ensemble แต่ถ้าเป้าหมายคือ accuracy โดยรวมไม่คุ้ม

> Ensemble ตัวนี้ต้องแก้ปัญหาคำเข้ากันได้ก่อนจึงจะรันได้: StackingClassifier ฝึก base learner ซ้ำบน CV fold และเมื่อคลาส SQL Injection มีแถวเดียว fold ส่วนใหญ่จะไม่มีคลาสนี้ ทำให้ y ขาดช่วง (0..12, 14) ซึ่ง XGBoost ปฏิเสธ label ที่ไม่ต่อเนื่องทันทีและทำให้ทั้ง ensemble ล้ม คลาส LabelSafeXGBClassifier ใน src/ids2018/models.py จึงเข้ารหัส label ใหม่ให้ต่อเนื่องก่อนฝึก แล้วรายงานรหัสเดิมผ่าน classes_ เพื่อให้ scikit-learn เต็มคอลัมน์ความน่าจะเป็นที่ขาดหายไปเอง

4. ตารางสรุปการแลกเปลี่ยน

4.1 การเปรียบเทียบหลัก

โมเดล	Accuracy	F1-macro	Recall-macro	Precision-macro	MCC	Infiltration F1
Stacking	0.9839	0.6886	0.6806	0.7203	0.9465	0.1504
LightGBM	0.9837	0.6870	0.6797	0.7174	0.9460	0.1328
Random Forest	0.9828	0.6869	0.6851	0.7172	0.9431	0.1037
XGBoost	0.9837	0.6854	0.6796	0.7150	0.9460	0.0963
CatBoost	0.9836	0.6764	0.6712	0.7132	0.9454	0.0174
MLP	0.9816	0.6698	0.6728	0.6742	0.9389	0.0000
Logistic Regression	0.9651	0.6296	0.6220	0.6599	0.8823	0.0193

4.2 การเปรียบเทียบต้นทุนเชิงปฏิบัติการ

โมเดล	เวลาฝึก (วิ)	อุปกรณ์	Inference (flow/วิ)	ขนาดไฟล์ (MB)	เดือนฝึก (FP)	โจมตีที่พลาด (FN)
CatBoost	12.9	GPU	1,248,090	1.64	46	914

โมเดล	เวลาฝึก (วิ)	อุปกรณ์	Inference (flow/วิ)	ขนาดไฟล์ (MB)	เดือนฝึก (FP)	โจมตีที่พลาด (FN)
Random Forest	20.4	CPU	106,908	29.04	149	874
LightGBM	23.1	CPU	34,527	6.43	104	842
XGBoost	29.2	GPU	119,092	3.46	88	860
MLP	51.0	CPU	1,542,416	0.21	96	1,004
Logistic Regression	66.6	CPU	6,579,284	0.0045	609	1,912
Stacking	118.2	GPU	30,881	14.24	101	834

ขนาดไฟล์นี้เฉพาะสำเนา .joblib ซึ่งทุกโมเดลมีเหมือนกัน ส่วนไฟล์รูปแบบเฉพาะของแต่ละไลบรารี (xgboost.json, catboost.cbm, lightgbm.txt) ถูกบันทึกควบคู่ไปด้วยแต่ไม่นับรวมในตารางนี้

เวลารวมทั้งหมด: 8.3 นาที สำหรับ 7 โมเดล รวมการดึงตัวอย่างแบบ 2 pass จาก CSV ขนาด 6.7 GB ด้วย

5. การประเมินรายการคลาสการโจมตี

5.1 ตาราง F1 รายการคลาสแบบเต็ม

คลาส	จำนวน	Stacking	LightGBM	RF	XGBoost	CatBoost	MLP	LogReg
Benign	74,764	0.9938	0.9937	0.9932	0.9937	0.9936	0.9927	0.9833
DDOS attack-HOIC	3,803	0.9999	0.9997	0.9955	1.0000	1.0000	0.9974	0.9596
DDoS attacks-LOIC-HTTP	3,194	0.9994	0.9994	0.9969	0.9992	0.9994	0.9907	0.9619
DoS attacks-Hulk	2,561	1.0000	0.9998	0.9984	1.0000	0.9981	0.9971	0.9959
Bot	1,587	0.9934	0.9934	0.9817	0.9937	0.9909	0.9553	0.6061
FTP-Brute Force	1,072	0.7884	0.7871	0.7870	0.7884	0.7875	0.7799	0.7438
SSH-Brute force	1,040	1.0000	1.0000	1.0000	1.0000	1.0000	0.9990	0.9924
Infiltration	898	0.1504	0.1328	0.1037	0.0963	0.0174	0.0000	0.0193
DoS attacks-SlowHTTPTest	776	0.5987	0.5967	0.5950	0.5987	0.5931	0.6080	0.5795
DoS attacks-GoldenEye	230	0.9978	0.9957	0.9957	0.9957	0.9935	0.9891	0.9690
DoS attacks-Slowloris	61	0.9833	0.9833	0.9672	0.9917	0.8829	0.8807	0.8333

คลาส	จำนวน	Stacking	LightGBM	RF	XGBoost	CatBoost	MLP	LogReg
DDOS attack-LOIC-UDP	9	0.8235	0.8235	0.8889	0.8235	0.8889	0.8571	0.8000
Brute Force -Web	3	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Brute Force -XSS	1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
SQL Injection	1	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

5.2 การโจมตีปริมาณสูง — แก้ได้แล้ว

DDOS-HOIC, DDoS-LOIC-HTTP, DoS-Hulk, SSH-Bruteforce และ DoS-GoldenEye ได้ F1 ตั้งแต่ 0.99 ขึ้นไปในโมเดลที่ไม่เป็นเชิงเส้นทั้ง 6 ตัว โดยมี 4 คะแนนที่ได้ 1.0000 พอดี ส่วน Bot ได้ 0.99+ ในโมเดลต้นไม้ทุกตัว การโจมตีตระกูล volumetric และ brute-force เหล่านี้แยกออกได้สมบูรณ์ด้วยคุณลักษณะจาก CICFlowMeter จึงไม่มีประโยชน์ในการใช้เลือกโมเดล เพราะทุกโมเดลทำได้ถูกหมด

5.3 สามคลาสที่เรียนรู้ไม่ได้

Brute Force -Web (3 flow ในชุดทดสอบ), Brute Force -XSS (1) และ SQL Injection (1) ได้ 0.0000 พอดีในทั้ง 7 โมเดล ในระดับ corpus คลาสเหล่านี้มี 611, 230 และ 87 แถวจากทั้งหมด 16.2 ล้านแถว การสุ่มแบบ stratified ตามสัดส่วนที่ 1.85% จึงได้ 11, 4 และ 2 แถว และเมื่อแบ่ง 70/30 เหลืออยู่ในชุดทดสอบเพียง 1-3 แถว

ห้ามสรุปเรื่องคุณภาพของโมเดลจากข้อมูล 3 แถวนี้นะ ควรรายงานว่า "ข้อมูลไม่เพียงพอ" มากกว่ารายงานเป็นคะแนนศูนย์ การเพิ่มค่า --min-per-class จะทำให้คลาสเหล่านี้มีข้อมูลพอใช้ แต่จะทำลายความเป็นสัดส่วนของ stratification ที่ส่วนอื่นของงานนี้พึ่งพาอยู่

5.4 Infiltration — คลาสที่ตัดสินอันดับ

โมเดล	Infiltration F1	F1-macro	อันดับที่ตรงกัน
Stacking	0.1504	0.6886	1 / 1
LightGBM	0.1328	0.6870	2 / 2
Random Forest	0.1037	0.6869	3 / 3
XGBoost	0.0963	0.6854	4 / 4
CatBoost	0.0174	0.6764	5 / 5
MLP	0.0000	0.6698	6 / 6

สำหรับโมเดลที่ไม่เป็นเชิงเส้นทั้ง 6 ตัว อันดับของ Infiltration กับอันดับของ F1-macro ตรงกันทุกตำแหน่ง โดย Infiltration อธิบายช่องว่างระหว่าง Stacking กับ MLP ได้ 53% และระหว่าง Stacking กับ CatBoost ได้ 73%

ความล้มเหลวเบื้องหลังเห็นได้จาก confusion matrix ของ XGBoost: flow ปกติ 6,490 จาก 74,764 (8.7%) ถูกทำนายเป็น Infiltration และ flow ที่เป็น Infiltration จริง 518 จาก 898 ถูกทำนายเป็น Benign ในคอนฟิกรูที่ถ่วงน้ำหนักคลาส precision ขึ้นได้เพียง 0.055 ที่ recall 0.422 กล่าวคือ ทราฟฟิกการแทรกซึมใน

CSE-CIC-IDS2018 ในระดับ flow feature นั้นแยกไม่ออกจากกราฟฟิคภายในองค์กรทั่วไป ซึ่งเป็นข้อจำกัดของตัวการเก็บข้อมูลเอง และมีรายงานอย่างกว้างขวางในวรรณกรรม

5.5 การชนกันของ SlowHTTPTest กับ FTP-BruteForce

DoS-SlowHTTPTest ทำได้สูงสุด F1 0.6080 (MLP) และ FTP-BruteForce สูงสุด 0.7884 (Stacking/XGBoost) โดยช่วงความต่างระหว่างทั้ง 7 โมเดลอยู่ที่เพียง 0.029 และ 0.045 ตามลำดับ confusion matrix ของ XGBoost แสดงว่า SlowHTTPTest 391 จาก 776 flow ถูกทำนายเป็น FTP-BruteForce และ FTP-BruteForce 126 จาก 1,072 flow ถูกทำนายเป็น SlowHTTPTest

การโจมตีทั้งสองแบบสร้างการเชื่อมต่ออายุสั้นจำนวนมากไปยังบริการเดียวกัน และเมื่อตัด Dst Port กับ Timestamp ออกไปในฐานะ leakage แล้ว สถิติ flow ที่เหลือก็ไม่เพียงพอจะแยกทั้งสองออกจากกัน การเก็บ Dst Port ไว้จะแยกได้เกือบสมบูรณ์ — และนั่นคือ leakage เพราะมันเข้ารหัสวิธีการเชื่อมต่อ ไม่ใช่พฤติกรรมโจมตี

> ไม่ได้ทำการวิเคราะห์ SHAP สำหรับชุดข้อมูลนี้ หลักฐานเรื่องความสอดคล้องข้ามโมเดลข้างต้นจึงมาจาก confusion matrix ไม่ใช่จากการวิเคราะห์ feature attribution

6. ข้อเสนอแนะสำหรับการนำไปใช้งานจริง

ไม่มีผู้ชนะเพียงหนึ่งเดียวในทุกแกน ข้อเสนอแนะจึงระบุแยกตามแกน พร้อมบอกการแลกเปลี่ยนอย่างชัดเจน

ตัวตรวจจับหลักแบบ inline: CatBoost เดือนผิด 46 ครั้ง (binary FPR 0.00062 ดีที่สุดในสนามด้วยระยะห่าง 2 เท่า), 1,248,090 flow/วินาที (เร็วที่สุดในกลุ่มโมเดลที่แม่นยำ ด้วยระยะห่าง 10 เท่า), ผิก 12.9 วินาที, ไฟล์ 1.64 MB ต้นทุนคือพลาดการโจมตีเพิ่ม 80 ครั้งเทียบกับ Stacking (+0.5 จุดเปอร์เซ็นต์ของ flow โจมตีทั้งหมด) และตรวจ Infiltration แทบไม่ได้เลย — ซึ่งเมื่อพิจารณาว่าโมเดลที่ดีที่สุดในสนามก็ทำได้แค่ F1 0.1504 บนคลาสสิน ก็ไม่ใช่ความสามารถที่โมเดลใดในที่นี้มีให้อยู่แล้ว

ทางเลือกที่สมดุล / สำหรับไซต์ที่มีแค่ CPU: XGBoost อัตราเดือนผิดต่ำเป็นอันดับสอง (88 FP), F1-macro อันดับ 4, 119,092 flow/วินาที, ไฟล์ 3.46 MB ไม่เคยดีที่สุดในแกนใด แต่ก็ไม่เคยต่ำกว่าอันดับ 4 ในแกนใด

งานวิเคราะห์แบบ batch หรือ forensic ที่ไม่สนความเร็ว: LightGBM F1-macro ดีที่สุดในโมเดลเดียว (0.6870) และ Infiltration F1 ดีเป็นอันดับสอง (0.1328) แต่ความเร็ว 34,527 flow/วินาทีทำให้ใช้แบบ inline ไม่ได้

ใช้ Stacking เฉพาะเมื่อ Infiltration เป็นเป้าหมายชัดเจน มันดีที่สุดในสนามทั้ง accuracy, F1-macro และจำนวนการโจมตีที่พลาด แต่ระยะห่างจาก LightGBM คือ +0.0002, +0.0016 และ 8 flow ตามลำดับ ด้วยราคาเวลาผิก 5.1 เท่าและความเร็วที่แย่ที่สุดเท่าที่วัดได้

อย่าใช้ Logistic Regression เป็นตัวตรวจจับหลัก เดือนผิด 609 ครั้ง (6.6 เท่าของโมเดลที่แนะนำ) และพลาดการโจมตี 1,912 ครั้ง (2.3 เท่าของตัวที่ดีที่สุด) แต่ยังใช้เป็นตัวกรองขั้นแรกราคาถูกได้ที่ 6.6 ล้าน flow/วินาที บนไฟล์ 4.5 KB

อย่าพึ่งพาโมเดลใดในงานนี้สำหรับ Brute Force -Web, Brute Force -XSS, SQL Injection หรือ Infiltration สามตัวแรกไม่มีข้อมูลพอจะวัดผลได้ ส่วนตัวที่สี่เรียนรู้ไม่ได้จากคุณลักษณะชุดนี้ ควรครอบคลุมการโจมตีตระกูลเหล่านี้ด้วยระบบแบบ signature-based หรือ host-based แทน

7. ข้อจำกัดและภัยคุกคามต่อความถูกต้อง

1. การแบ่งข้อมูลเป็นแบบสุ่ม ไม่ใช่ตามลำดับเวลา นี่เป็นข้อจำกัดที่ร้ายแรงที่สุด

และเป็นความแตกต่างที่ใหญ่ที่สุดจากงาน CICIDS2017 ที่เป็นคู่ขนานกัน

ซึ่งใช้โปรโตคอลตามลำดับเวลาแบบเข้มงวดที่ปราศจาก leakage การแบ่งแบบสุ่มยอมให้ flow จากการโจมตีชุดเดียวกัน — ซึ่งมักเป็นข้อมูลที่แทบซ้ำกัน

เกิดขึ้นหากันไม่กี่วินาทีจากเครื่องมือเดียวกันยังไปตีเป้าหมายเดียวกัน — ปรากฏทั้งในชุดฝึกและชุดทดสอบ

คะแนนที่รายงานทั้งหมดจึงมองโลกในแง่ดีเกินจริง และควรอ่านเป็นขอบเขตบน มันวัดการ interpolate ภายในการเก็บข้อมูลครั้งเดียว ไม่ได้วัดการ generalize ไปยังกราฟฟิคในอนาคต

2. ไม่ได้ลบ flow ที่ซ้ำกัน ตัวเลือก --drop-duplicates ถูกปิดไว้เป็นค่าเริ่มต้น เพราะการเปิดใช้จะทำให้จำนวนแถวสุดท้ายเปลี่ยนและทำลายโครงสร้าง 300,000/210,000/90,000 ที่กำหนดไว้ เมื่อรวมกับข้อจำกัดที่ 1 flow ที่ซ้ำกันเป๊ะและคร่อมเส้นแบ่ง train/test จะทำให้ทุกคะแนนในเอกสารนี้สูงเกินจริง

3. สามคลาสมีข้อมูลทดสอบเพียง 1–3 flow Brute Force -Web, Brute Force -XSS และ SQL Injection ไม่สามารถใช้สรุปอะไรได้ คะแนน 0.0000 ของพวกมันเป็นผลจากการมีข้อมูลน้อย ไม่ใช่การวัดความสามารถ

4. ไม่ได้จูน hyperparameter ให้โมเดลใดเลย ข้อนี้เป็นข้อดีในแง่ความเป็นธรรม เพราะไม่มีโมเดลใดได้เปรียบ แต่ก็หมายความว่าทุกคะแนนที่รายงานเป็นค่าต่ำสุดที่เป็นไปได้ และอันดับอาจเปลี่ยนหากจูนแยกรายโมเดล อีกทั้งยังทำให้เทียบกับตัวเลขในเปเปอร์ที่จูนมาแล้วไม่ได้

5. การเทียบเวลาฝึกถูกรบกวนจากฮาร์ดแวร์ XGBoost, CatBoost และ Stacking ใช้ CUDA ส่วน Random Forest, LightGBM, MLP และ Logistic Regression ใช้ CPU ดู §2.1

6. ชุดข้อมูลเดียว ตัวอย่างชุดเดียว การแบ่งครั้งเดียว seed เดียว ผลทั้งหมดมาจาก RANDOM_STATE=42 บนการสุ่ม 300,000 แถวเพียงครั้งเดียว ไม่มี cross-validation ไม่มีการสุ่มซ้ำ จึงไม่มีช่วงความเชื่อมั่น ความต่างของ F1-macro ที่น้อยกว่าประมาณ 0.005 — ซึ่งครอบคลุมโมเดล 4 อันดับแรก — ไม่ควรถือว่าเป็นความต่างจริง

7. ตัวอย่างคิดเป็นเพียง 1.85% ของ corpus ยังไม่ได้ทดสอบว่าอันดับจะคงเดิมหรือไม่ที่ 500,000 หรือ 1,000,000 แถว การออกแบบการสุ่มเป็นแบบซ้อนกัน (300k □ 500k □ 1M) จึงวัดเรื่องนี้ได้โดยตรงด้วยการรันซ้ำและเปลี่ยน --sample-size

8. ไม่ได้ทำการวิเคราะห์ SHAP หรือ feature attribution ข้ออ้างเรื่อง *สาเหตุ* ที่โมเดลต่าง ๆ ผลิตผลตรงกันจึงอาศัยหลักฐานจาก confusion matrix เพียงอย่างเดียว

9. ผลของ LightGBM ขึ้นอยู่กับพารามิเตอร์ที่ไม่ใช่ค่าเริ่มต้น ดู §2.1 การทำซ้ำด้วยค่าเริ่มต้นของไลบรารีจะได้โมเดลที่ฟังแบบเงิบ ๆ

ที่มาของข้อมูล / Data Provenance

ไฟล์	เนื้อหา
results/ids2018/300k/model_comparison.csv	ตารางเปรียบเทียบหลัก 7 โมเดล
results/ids2018/300k/extended_metrics.csv	MCC, binary FPR, FP/FN, throughput, ขนาดไฟล์
results/ids2018/300k/per_class_f1_matrix.csv	ตาราง F1 รายคลาส
results/ids2018/300k/<model>/confusion_matrix.csv	Confusion matrix รายโมเดล
results/ids2018/300k/<model>/per_class_report.csv	precision/recall/F1/support รายคลาส
results/ids2018/300k_balanced_weighting/	ผลการรันคู่ขนานแบบถ่วงน้ำหนักคลาส
models/ids2018/300k/metadata.json	รายชื่อ feature, คอลัมน์ที่ตัด, คลาส, seed, คอนฟิก

คำสั่งที่ใช้สร้างผลชุดนี้ / Command used to produce this bundle:

```
python -m src.ids2018.train_ids2018 --sample-size 300000 --chunksize 300000 \
    --class-weighting none --accelerator gpu --gpu-devices 0
```